

Experiment 6

Decision Tree

Aim:

To implement the Decision Tree algorithm for classification and analyze its performance.

Theory:

A Decision Tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure consisting of a root node, internal nodes, branches, and leaf nodes.

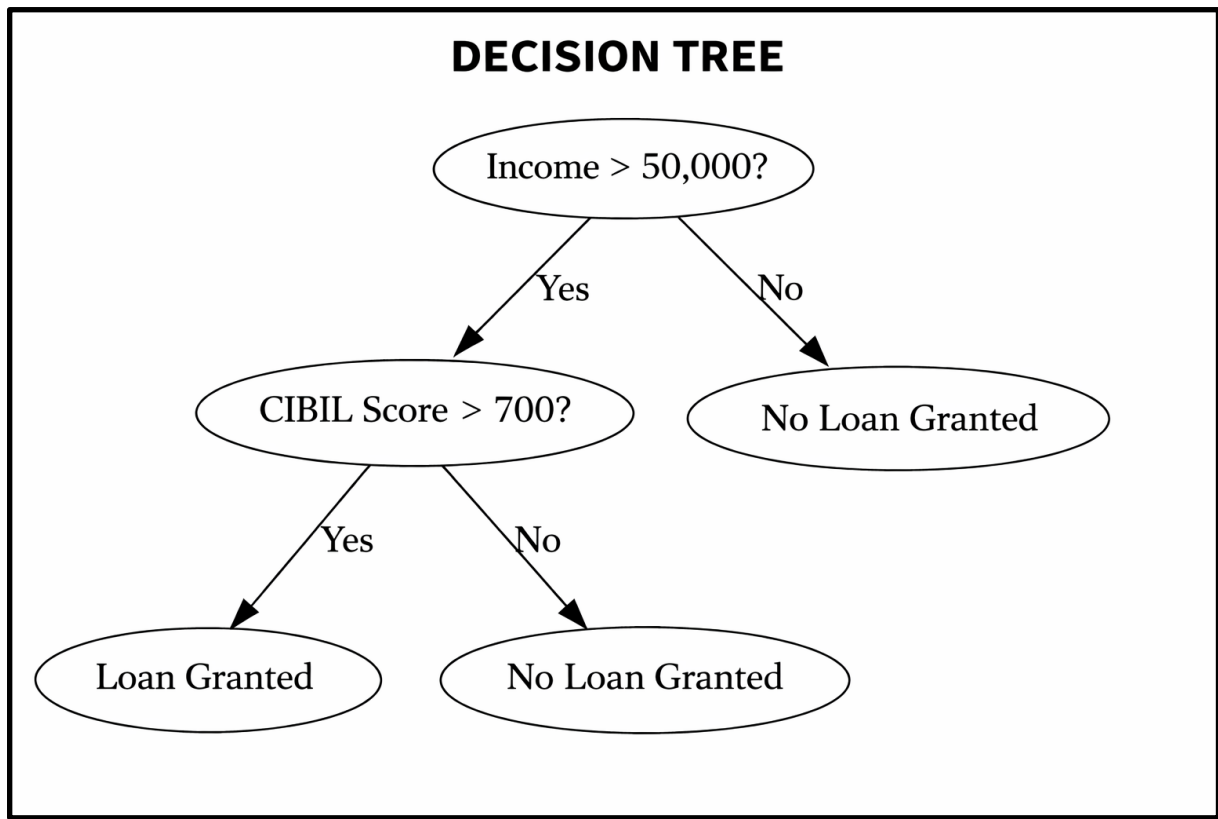
The main components of a Decision Tree are:

- Internal Nodes: Represent attribute tests
- Branches: Represent outcomes of attribute tests.
- Leaf Nodes: Represent final decision or class labels.

As shown in the below figure, a Decision Tree works by recursively subdividing the dataset into smaller groups based on feature values. At each step, the algorithm selects the best feature that separates the data into different classes using an appropriate splitting criterion. This splitting process continues until the data within each node becomes highly uniform or a predefined stopping condition is met. Internal nodes represent decision conditions used to split the data, while leaf nodes represent the final predicted class.

To determine the best split at each node, Decision Trees commonly use Gini Index and Entropy as splitting criteria. The Gini Index measures the impurity of a node and represents the probability of incorrectly classifying a randomly chosen sample if it were labeled according to the class distribution in that node. Lower Gini values indicate purer nodes, and the Gini value becomes zero when all samples belong to a single class. However, it is slightly biased toward dominant classes.

Entropy, on the other hand, measures the amount of uncertainty or disorder in a node and is derived from information theory. It is used to calculate Information Gain, which helps in selecting the most informative feature for splitting. Entropy is zero for a perfectly pure node, while higher entropy values indicate greater disorder. It is sensitive to small changes in class distribution and often produces balanced and informative splits.



Merits of Decision Tree

1. **Easy to Understand and Interpret** – The tree structure is simple and intuitive, making it easy to visualize and explain decisions.
2. **Handles Both Numerical and Categorical Data** – Decision Trees can work with different types of data without much preprocessing.
3. **No Need for Feature Scaling** – Normalization or standardization is not required, as trees are not affected by feature scaling.

Demerits of Decision Tree

1. **Prone to Overfitting** – A deep decision tree may fit noise in the training data, reducing generalization.
2. **Unstable Model** – Small changes in data can lead to a completely different tree structure.

3. **Lower Accuracy Compared to Ensembles** – Single decision trees often perform worse than ensemble methods like Random Forest.

INPUT: Import Libraries

```
# Import necessary libraries
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
```

```
from sklearn.preprocessing import OneHotEncoder
```

```
from sklearn.compose import ColumnTransformer
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
print("Libraries imported successfully!")
```

OUTPUT:

```
Libraries imported successfully!
```

INPUT: Load Dataset

```
# Load the dataset
```

```
data = pd.read_csv("/content/loan-approval-prediction-  
dataset/loan_approval_dataset.csv")
```

```
print("Dataset loaded successfully")
```

OUTPUT:

Dataset loaded successfully!

INPUT: Check the shape of the dataset

```
# Check the shape of the dataset
```

```
print("Dataset Shape:", data.shape)
```

OUTPUT:

Dataset Shape: (4269, 13)

INPUT: Print columns

```
# Fix column spacing
```

```
data.columns = data.columns.str.strip()
```

```
# Print columns in vertical form
```

```
print("\nColumns (Vertical List):")
```

```
for col in data.columns:
```

```
    print(col)
```

OUTPUT:

Columns (Vertical List):

loan_id

no_of_dependents

Education

self_employed

Income_annum

loan_amount

loan_term

cibil_score

residential_assets_value

commercial_assets_value

luxury_assets_value

bank_asset_value loan_status

INPUT: Display first 5 rows

Display first 5 rows

print("First 5 rows of the dataset:")

data.head()

OUTPUT:

First 5 rows of the dataset:

l o a n _ i d	no_ of_ dep end ents	e d uc at ion	self _e mp loy ed	inc om e_a nn um	loa n_ am ou nt	lo an _t er m	ci bil _s co re	resid ential _asse ts_val ue	com merci al_ass ets_v alue	luxu ry_a sset s_va lue	ban k_a sset _val ue	lo an _s ta tu s		l o a n _ i d	no_ of_d epe nde nts
0	1	2	Gr ad uat e	No	96 00 00 00	29 90 00 00	12	778	2400 000	176 000 000	227 000 000	80 00 00 0	A p pr o ve d	0	1

1	2	0	No t Gr ad uat e	Yes	41 00 00 0	12 20 00 00	8	417	2700 000	220 000 0	880 000 0	33 00 00 0	R ej ec te d	1	2
2	3	3	Gr ad uat e	No	91 00 00 0	29 70 00 00	20	506	7100 000	450 000 0	333 000 00	12 80 00 00	R ej ec te d	2	3
3	4	3	Gr ad uat e	No	82 00 00 0	30 70 00 00	8	467	1820 0000	330 000 0	233 000 00	79 00 00 0	R ej ec te d	3	4
4	5	5	No t Gr ad uat e	Yes	98 00 00 0	24 20 00 00	20	382	1240 0000	820 000 0	294 000 00	50 00 00 0	R ej ec te d	4	5

INPUT: Encode Categorical Variables

Target Column Encoding

```
data.columns = data.columns.str.strip()
```

```
data['loan_status'] = data['loan_status'].str.strip().map({
```

```
    'Approved': 1,
```

```
    'Rejected': 0
```

```
})
```

```
print("Target column loan_status encoded successfully")
```

```
data.head()
```

OUTPUT:

Target column loan_status encoded successfully

l o a n _i d	no_ of_d epe nde nts	e d uc at ion	self_ e mp loy ed	inc om e_a nnu m	loa n_ am ou nt	lo an _t er m	ci bil _s co re	resid ential _asse ts_val ue	comm ercial _asset s_valu e	luxu ry_a ssets _val ue	ban k_a sset _val ue	lo an _s tat us		l o a n _i d	no_ of_d epe nde nts
0	1	2	Gra du ate	No	96 00 00 0	29 90 00 00	12	778	24000 00	1760 0000	227 000 00	80 00 00 0	1	0	1
1	2	0	No t Gra du ate	Yes	41 00 00 0	12 20 00 00	8	417	27000 00	2200 000	880 000 0	33 00 00 0	0	1	2
2	3	3	Gra du ate	No	91 00 00 0	29 70 00 00	20	506	71000 00	4500 000	333 000 00	12 80 00 00	0	2	3
3	4	3	Gra du ate	No	82 00 00 0	30 70 00 00	8	467	18200 000	3300 000	233 000 00	79 00 00 0	0	3	4
4	5	5	No t Gra du ate	Yes	98 00 00 0	24 20 00 00	20	382	12400 000	8200 000	294 000 00	50 00 00 0	0	4	5

INPUT: Data Information

```
#Data Information
```

```
print("All Required Information Related To Data: ")
```

```
data.info()
```

OUTPUT:

Data columns (total 13 columns):

```
# Column Non-Null Count Dtype
```

0 loan_id 4269 non-null int64

1 no_of_dependents 4269 non-null int64

2 education 4269 non-null object

3 self_employed 4269 non-null object

4 income_annum 4269 non-null int64

5 loan_amount 4269 non-null int64

6 loan_term 4269 non-null int64

7 cibil_score 4269 non-null int64

8 residential_assets_value 4269 non-null int64

9 commercial_assets_value 4269 non-null int64

10 luxury_assets_value 4269 non-null int64

11 bank_asset_value 4269 non-null int64

12 loan_status 4269 non-null int64

INPUT: Check for missing (null) values

```
# Check for missing (null) values in each column
```

```
print("Missing values in each column:")
```

```
print(data.isnull().sum())
```

OUTPUT:

Missing values in each column:

loan_id 0

no_of_dependents 0

education 0

self_employed 0

income_annum 0

loan_amount 0

loan_term 0

cibil_score 0

residential_assets_value 0

commercial_assets_value 0

luxury_assets_value 0

bank_asset_value 0

loan_status 0

dtype: int64

INPUT: Data Transformation

Define target column

target_col = "loan_status"

print("Target column selected:", target_col)

Separate features and target

X = data.drop(columns=[target_col])

y = data[target_col]

```
print("Features (X) and target (y) separated successfully")

# Identify categorical columns

cat_cols = X.select_dtypes(include=['object']).columns.tolist()

print("Categorical columns identified:", cat_cols)

# Apply One-Hot Encoding to categorical columns

from sklearn.compose import ColumnTransformer

from sklearn.preprocessing import OneHotEncoder

preprocessor = ColumnTransformer(

    transformers=[

        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)

    ],

    remainder='passthrough'

)

# Transform feature data

X_transformed = preprocessor.fit_transform(X)

print("Categorical features one-hot encoded and dataset transformed successfully")
```

OUTPUT:

Target column selected: loan_status

Features (X) and target (y) separated successfully

Categorical columns identified: ['education', 'self_employed']

Categorical features one-hot encoded and dataset transformed successfully

INPUT: Split the Data

```
# Split the dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split( X_transformed, y, test_size=0.2,  
random_state=42, stratify=y)
```

```
print("Dataset successfully split into training and testing sets")
```

```
print("Training set size:", X_train.shape)
```

```
print("Testing set size:", X_test.shape)
```

OUTPUT

Dataset successfully split into training and testing sets

Training set size: (3415, 14)

Testing set size: (854, 14)

GINI

INPUT: Initialize and train Decision Tree model

```
# Initialize and train Decision Tree model using Gini impurity
```

```
dt_model = DecisionTreeClassifier(criterion='gini', random_state=42)
```

```
dt_model.fit(X_train, y_train)
```

```
print("Decision Tree model trained successfully using Gini impurity")
```

OUTPUT:

Decision Tree model trained successfully using Gini impurity

INPUT: Evaluate Decision Tree model

```
# Evaluate Decision Tree model (Gini impurity)

y_pred = dt_model.predict(X_test)

print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix (Gini, Full Tree):")

print(confusion_matrix(y_test, y_pred))
```

OUTPUT:

Accuracy (Gini, Full Tree): 0.9789227166276346

Confusion Matrix (Gini, Full Tree):

```
[[313 10]
```

```
[ 8 523]]
```

INPUT: Compute confusion matrix

```
# Compute confusion matrix for Decision Tree model (Gini impurity)

print("Calculating confusion matrix (Gini)...")

cm = confusion_matrix(y_test, y_pred)

print("Confusion matrix calculated successfully")
```

OUTPUT:

Confusion matrix calculated successfully

INPUT: Plot confusion matrix

```
# Plot confusion matrix for Decision Tree model (Gini impurity)
```

```

print("Displaying confusion matrix plot (Gini)...")

plt.figure(figsize=(8, 6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=target_names,
            yticklabels=target_names)

plt.title('Confusion Matrix (Gini)')

plt.ylabel('True Label')

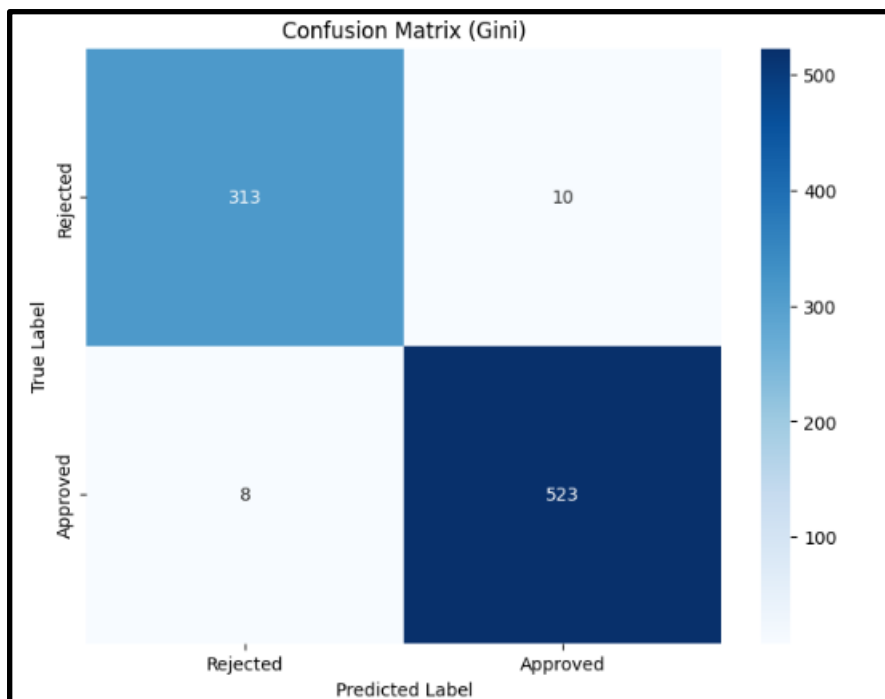
plt.xlabel('Predicted Label')

plt.show()

print("Confusion matrix plot displayed successfully")

```

OUTPUT:



ENTROPY

INPUT: Initialize and train Decision Tree model

```

# Train and evaluate Decision Tree model using Entropy

dt_model_entropy = DecisionTreeClassifier(criterion='entropy', random_state=42)

dt_model_entropy.fit(X_train, y_train)

print("Decision Tree model trained successfully using Entropy")

# Predict on test data

y_pred_entropy = dt_model_entropy.predict(X_test)

print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))

print("\nConfusion Matrix (Entropy, Full Tree):")

print(confusion_matrix(y_test, y_pred_entropy))

```

OUTPUT

Decision Tree model trained successfully using Entropy

Accuracy (Entropy, Full Tree): 0.9789227166276346

Confusion Matrix (Entropy, Full Tree):

```
[[312 11]
```

```
[ 7 524]]
```

INPUT: Compute confusion matrix

```

# Compute confusion matrix for Decision Tree model (Entropy)

print("Calculating confusion matrix (Entropy)...")

cm_entropy = confusion_matrix(y_test, y_pred_entropy)

print("Confusion matrix (Entropy) calculated successfully")

```

OUTPUT:

Confusion matrix (Entropy) calculated successfully

INPUT: Plot confusion matrix

```
# Plot confusion matrix for Decision Tree model (Entropy)
```

```
# Define target class names
```

```
target_names = ['Rejected', 'Approved']
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap( cm_entropy, annot=True, fmt='d', cmap='Greens', xticklabels=target_names,  
             yticklabels=target_names)
```

```
plt.title('Confusion Matrix (Entropy)')
```

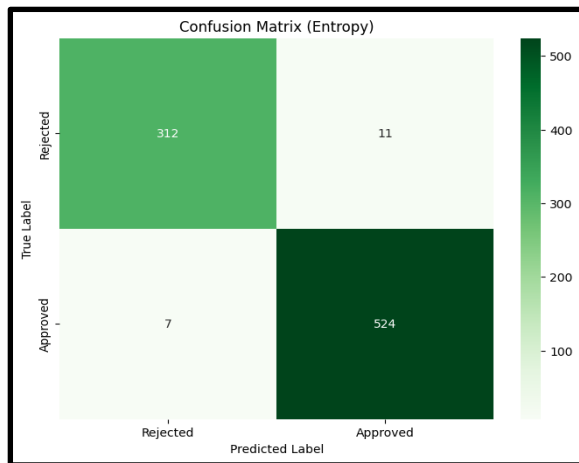
```
plt.ylabel('True Label')
```

```
plt.xlabel('Predicted Label')
```

```
plt.show()
```

```
print("Confusion matrix (Entropy) plot displayed successfully")
```

OUTPUT:



INPUT: GINI(MAX DEPTH: 3)

Train Decision Tree with max_depth=3

```
dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
```

```
dt_gini_3.fit(X_train, y_train)
```

Predictions

```
y_pred_3 = dt_gini_3.predict(X_test)
```

Accuracy

```
accuracy_3 = accuracy_score(y_test, y_pred_3)
```

```
print(f"Accuracy (Max Depth 3, Gini): {accuracy_3:.4f}")
```

Confusion Matrix

```
cm_3 = confusion_matrix(y_test, y_pred_3)
```

```
print("Confusion Matrix (Max Depth 3):")
```

```
print(cm_3)
```

Heatmap


```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(cm_3, annot=True, fmt='d', cmap='Blues', xticklabels=True, yticklabels=True)
```

```
plt.title("Confusion Matrix Heatmap (Max Depth 3, Gini)")
```

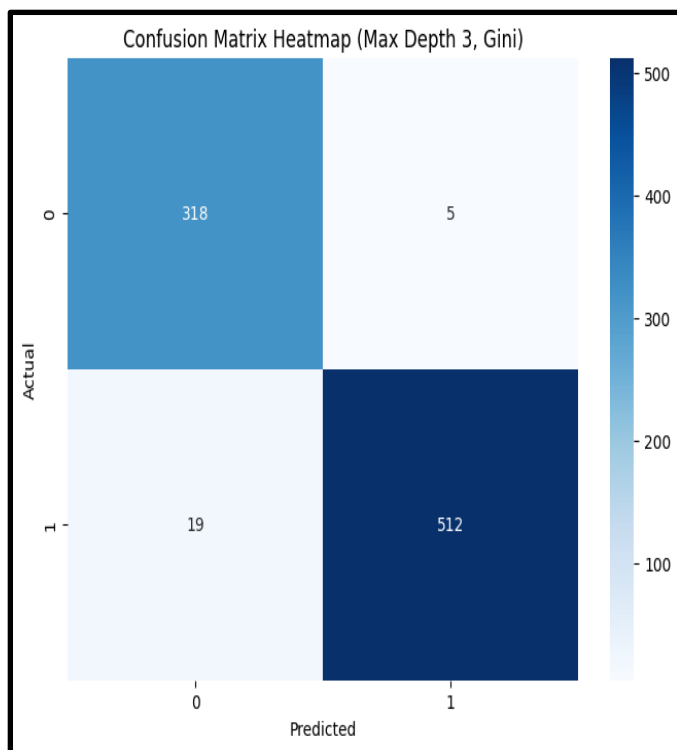
```
plt.xlabel("Predicted")
```

```
plt.ylabel("Actual")
```

```
plt.show()
```

OUTPUT:

Accuracy (Max Depth 3, Gini): 0.9719



INPUT: Plot Decision Tree

```
# Train Decision Tree with max_depth=3
```

```
dt_gini_3 = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=42)
```

```
dt_gini_3.fit(X_train, y_train)

# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(dt_gini_3, feature_names=None, class_names=None,

          filled=True,

          rounded=True,

          label='root',

          impurity=False,

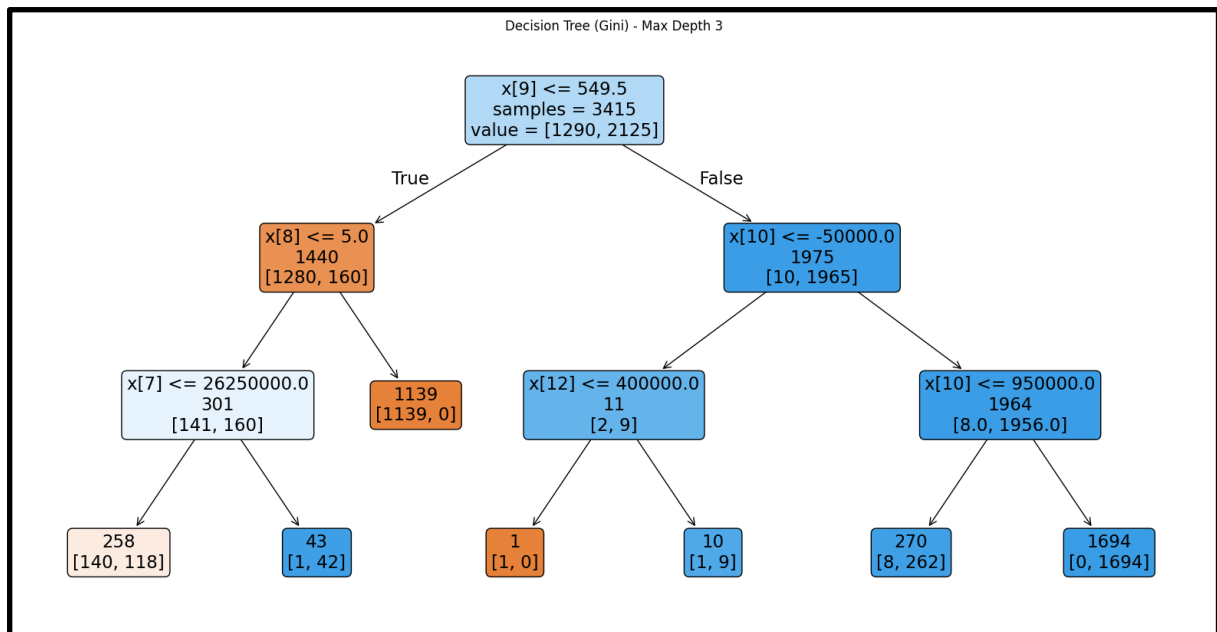
          proportion=False

)

plt.title("Decision Tree (Gini) - Max Depth 3")

plt.show()
```

OUTPUT:



INPUT: GINI(MAX DEPTH: 5)

Train Decision Tree with max_depth=5

```
dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)
```

```
dt_gini_5.fit(X_train, y_train)
```

Predictions

```
y_pred_5 = dt_gini_5.predict(X_test)
```

Accuracy

```
accuracy_5 = accuracy_score(y_test, y_pred_5)
```

```
print(f"Accuracy (Max Depth 5, Gini): {accuracy_3:.4f}")
```

Confusion Matrix

```
cm_5 = confusion_matrix(y_test, y_pred_5)
```

```
print("Confusion Matrix (Max Depth 5):")
```

```

print(cm_5)

# Heatmap

plt.figure(figsize=(8, 6))

sns.heatmap(cm_5, annot=True, fmt='d', cmap='Blues', xticklabels=True, yticklabels=True)

plt.title("Confusion Matrix Heatmap (Max Depth 5, Gini)")

plt.xlabel("Predicted")

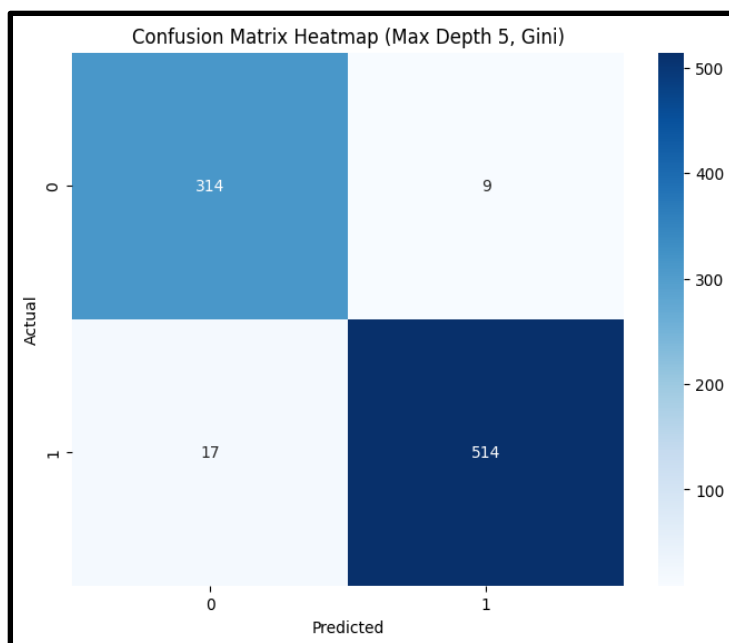
plt.ylabel("Actual")

plt.show()

```

OUTPUT:

Accuracy (Max Depth 5, Gini): 0.9696



INPUT: Plot Decision Tree

```
# Train Decision Tree with max_depth=5
```

```
dt_gini_5 = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)
```

```
dt_gini_5.fit(X_train, y_train)
```

```
# Plot the tree
```

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(
```

```
    dt_gini_5,
```

```
    feature_names=None,
```

```
    class_names=None,
```

```
    filled=True,
```

```
    rounded=True,
```

```
    label='root',
```

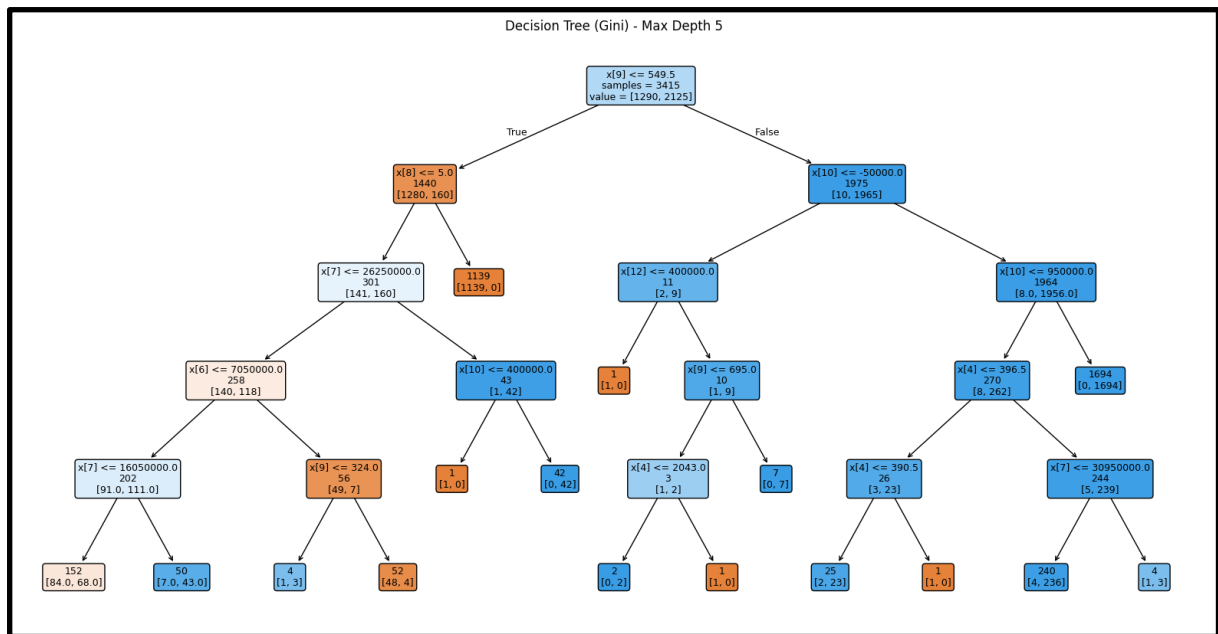
```
    impurity=False,
```

```
    proportion=False
```

```
)
```

```
plt.title("Decision Tree (Gini) - Max Depth 5")
```

```
plt.show()
```



INPUT: GINI(MAX DEPTH: 7)

Train Decision Tree with max_depth=7

```
dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)
```

```
dt_gini_7.fit(X_train, y_train)
```

Predictions

```
y_pred_7 = dt_gini_7.predict(X_test)
```

Accuracy

```
accuracy_7 = accuracy_score(y_test, y_pred_7)
```

```
print(f"Accuracy (Max Depth 7, Gini): {accuracy_7:.4f}")
```

Confusion Matrix

```
cm_7 = confusion_matrix(y_test, y_pred_7)
```

```
print("Confusion Matrix (Max Depth 7):")
```

```

print(cm_7)

# Heatmap

plt.figure(figsize=(8, 6))

sns.heatmap(cm_7, annot=True, fmt='d', cmap='Blues', xticklabels=True, yticklabels=True)

plt.title("Confusion Matrix Heatmap (Max Depth 7, Gini)")

plt.xlabel("Predicted")

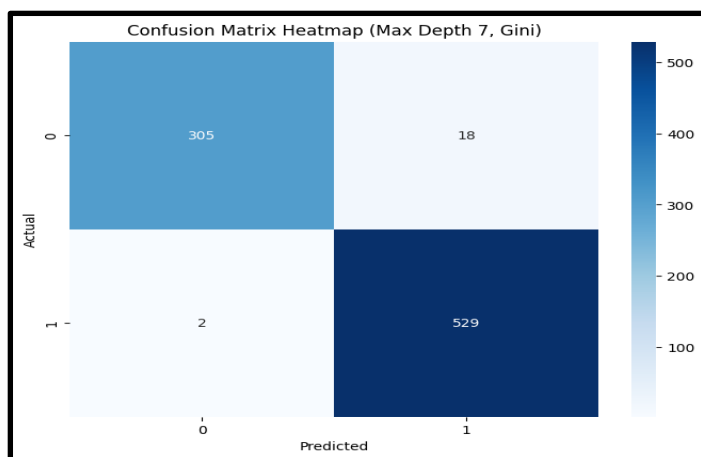
plt.ylabel("Actual")

plt.show()

```

OUTPUT:

Accuracy (Max Depth 7, Gini): 0.9766



INPUT : Plot The Decision Tree

```

# Train Decision Tree with max_depth=7

dt_gini_7 = DecisionTreeClassifier(criterion='gini', max_depth=7, random_state=42)

dt_gini_7.fit(X_train, y_train)

# Plot the tree

```


INPUT:ENTROPY (MAX DEPTH:3)

```
# Train Decision Tree with max_depth = 3 (Entropy)
```

```
dt_entropy_3 = DecisionTreeClassifier(
```

```
criterion='entropy',
```

```
max_depth=3,
```

```
random_state=42
```

```
)
```

```
dt_entropy_3.fit(X_train, y_train)
```

```
# Plot the tree
```

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(
```

```
dt_entropy_3,
```

```
feature_names=None,
```

```
class_names=None,
```

```
filled=True,
```

```
rounded=True,
```

```
label='root',
```

```
impurity=False,
```

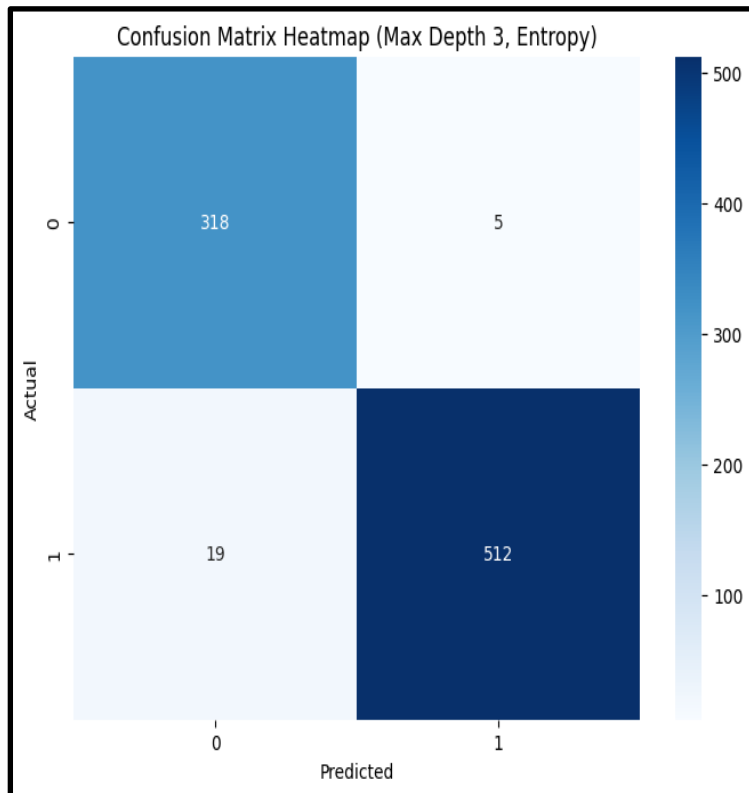
```
proportion=False
```

```
)
```

```
plt.title("Decision Tree (Entropy) - Max Depth 3")
```

```
plt.show()
```

OUTPUT:



INPUT : Plot The Decision Tree

```
# Train Decision Tree with max_depth = 3 (Entropy)
```

```
dt_entropy_3 = DecisionTreeClassifier(
```

```
criterion='entropy',
```

```
max_depth=3,
```

```
random_state=42
```

```
)
```

```
dt_entropy_3.fit(X_train, y_train)
```

```
# Plot the tree

plt.figure(figsize=(20, 10))

plot_tree(

    dt_entropy_3,

    feature_names=None,

    class_names=None,

    filled=True,

    rounded=True,

    label='root',

    impurity=False,

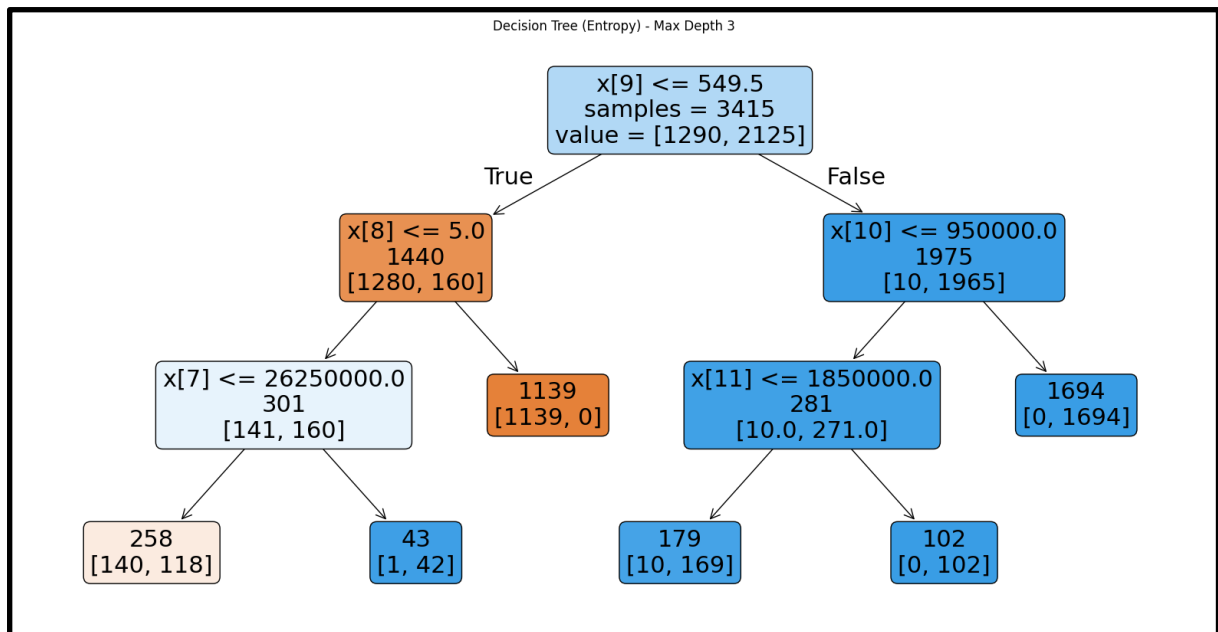
    proportion=False

)

plt.title("Decision Tree (Entropy) - Max Depth 3")

plt.show()
```

OUTPUT



INPUT:ENTROPY (MAX DEPTH:5)

Train Decision Tree with max_depth = 5 (Entropy)

```
dt_entropy_5 = DecisionTreeClassifier(
```

```
criterion='entropy',
```

```
max_depth=5,
```

```
random_state=42
```

```
)
```

```
dt_entropy_5.fit(X_train, y_train)
```

Plot the tree

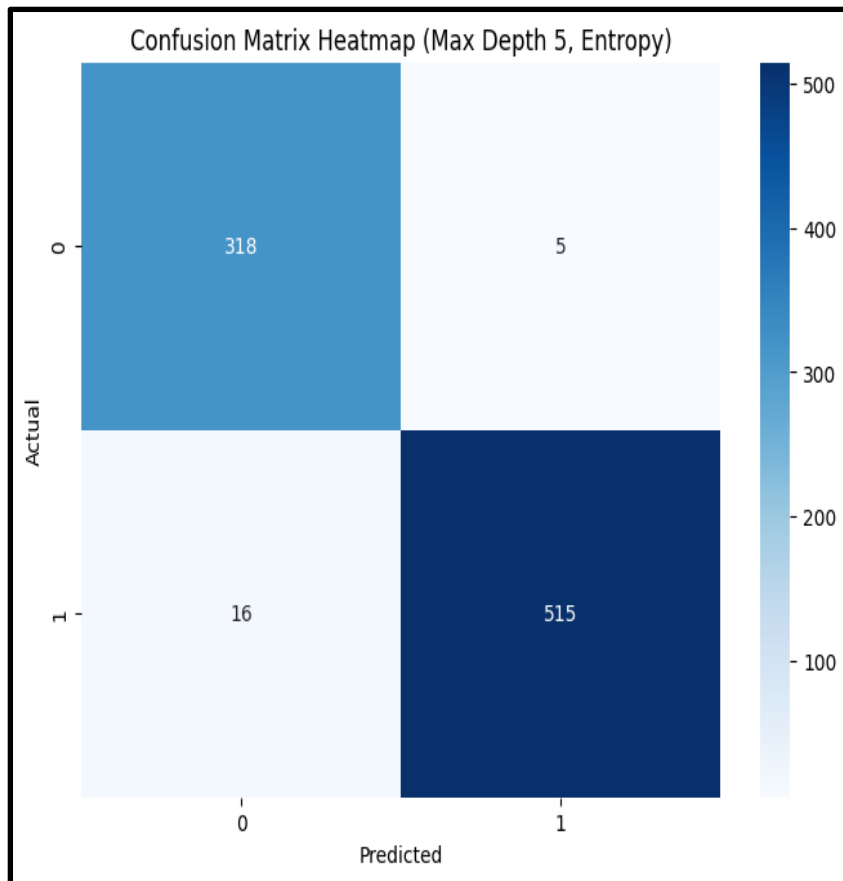
```
plt.figure(figsize=(20, 10))
```

```
plot_tree(
```

```
dt_entropy_5,
```

```
feature_names=None,  
  
class_names=None,  
  
filled=True,  
  
rounded=True,  
  
label='root',  
  
impurity=False,  
  
proportion=False  
  
)  
  
plt.title("Decision Tree (Entropy) - Max Depth 5")  
  
plt.show()
```

OUTPUT:



INPUT : Plot The Decision Tree

Train Decision Tree with max_depth = 5 (Entropy)

```
dt_entropy_5 = DecisionTreeClassifier(
```

```
criterion='entropy',
```

```
max_depth=5,
```

```
random_state=42
```

```
)
```

```
dt_entropy_5.fit(X_train, y_train)
```

Plot the tree

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(
```

```

dt_entropy_5,

feature_names=None,

class_names=None,

filled=True,

rounded=True,

label='root',

impurity=False,

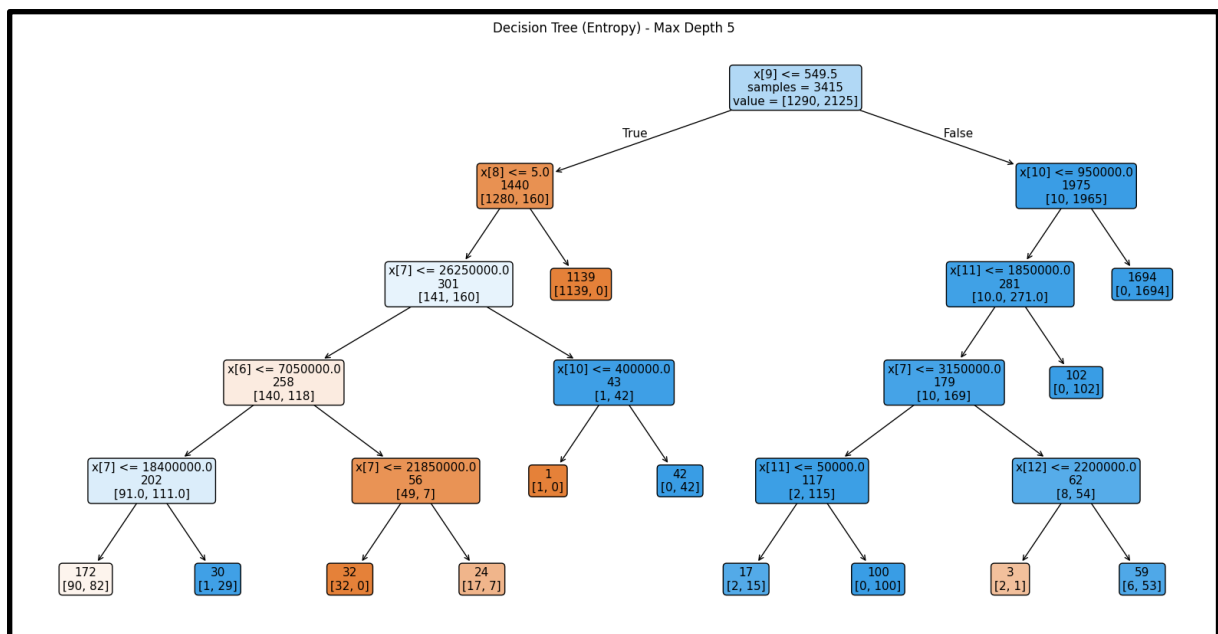
proportion=False

)

plt.title("Decision Tree (Entropy) - Max Depth 5")

plt.show()

```

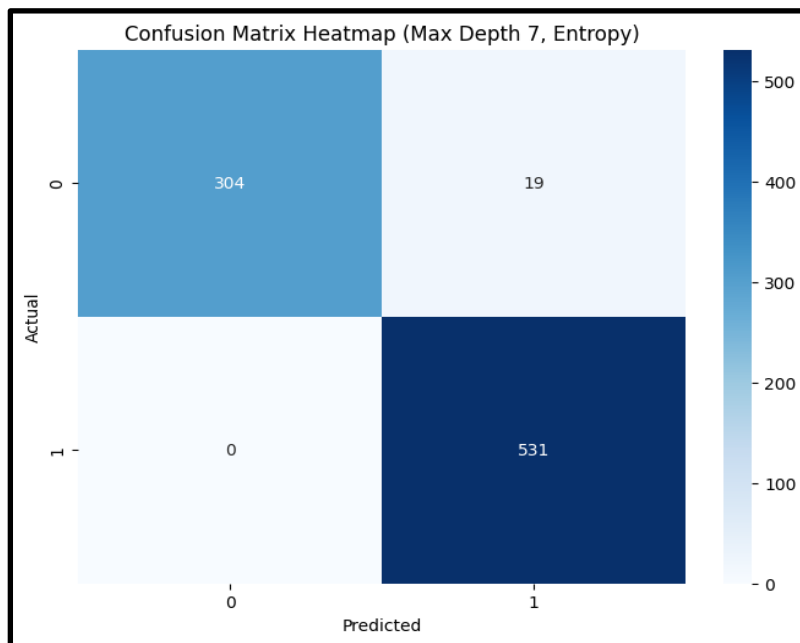


INPUT:ENTROPY (MAX DEPTH:7)

Train Decision Tree with max_depth = 7 (Entropy)

```
dt_entropy_7 = DecisionTreeClassifier(  
  
    criterion='entropy',  
  
    max_depth=7,  
  
    random_state=42  
  
)  
  
dt_entropy_7.fit(X_train, y_train)  
  
# Plot the tree  
  
plt.figure(figsize=(20, 10))  
  
plot_tree(  
  
    dt_entropy_7,  
  
    feature_names=None,  
  
    class_names=None,  
  
    filled=True,  
  
    rounded=True,  
  
    label='root',  
  
    impurity=False,  
  
    proportion=False  
  
)  
  
plt.title("Decision Tree (Entropy) - Max Depth 7")  
  
plt.show()
```


OUTPUT:



INPUT : Plot The Decision Tree

```
# Train Decision Tree with max_depth = 7 (Entropy)
```

```
dt_entropy_7 = DecisionTreeClassifier(
```

```
criterion='entropy',
```

```
max_depth=7,
```

```
random_state=42
```

```
)
```

```
dt_entropy_7.fit(X_train, y_train)
```

```
# Plot the tree
```

```
plt.figure(figsize=(20, 10))
```

```
plot_tree(
```

```
dt_entropy_7,
```

```
# Import necessary libraries
```

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn.preprocessing import LabelEncoder

print("Libraries imported successfully!")
```

OUTPUT:

Libraries imported successfully

INPUT: Load Dataset

```
# Load the dataset

data = pd.read_csv('ObesityDataSet_raw_and_data_synthetic.csv')

print("Dataset loaded successfully")
```

OUTPUT:

Dataset loaded successfully

INPUT: Check Shape of Dataset

```
# Check the shape of the dataset

print("Data Shape:")

print(data.shape)
```

OUTPUT:

Data Shape:(2111, 17)

INPUT: Display first 5 rows

```
# Display first 5 rows
```

```
print("First 5 rows of the dataset:")
```

```
data.head()
```

OUTPUT:

Gender Age Height Weight family_history_with_overweight FAVC FCVC NCP CAEC SMOKE
CH2O SCC FAF TUE CALC MTRANS NObeyesdad

0 Female 21.0 1.62 64.0 yes no 2.0 3.0 Sometimes no 2.0 no 0.0 1.0 no Sometimes
Public_Transportation Normal_Weight

1 Female 21.0 1.52 56.0 yes no 3.0 3.0 Sometimes yes 3.0 yes 3.0 0.0 Sometimes
Public_Transportation Normal_Weight

2 Male 23.0 1.80 77.0 yes no 2.0 3.0 Sometimes no 2.0 1.0 2.0 1.0 Frequently
Public_Transportation Normal_Weight

3 Male 27.0 1.80 87.0 no no 3.0 3.0 Sometimes no 2.0 2.0 2.0 0.0 Frequently Walking
Overweight_Level_I

4 Male 22.0 1.78 89.8 no no 2.0 1.0 Sometimes no 2.0 0.0 0.0 0.0 Sometimes
Public_Transportation Overweight_Level_II

INPUT: Display last 5 rows

```
# Display last 5 rows
```

```
print("Last 5 rows of the dataset:")
```

```
data.tail()
```

OUTPUT:

Gender Age Height Weight family_history_with_overweight FAVC FCVC NCP CAEC SMOKE
CH2O SCC FAF TUE CALC MTRANS NObeyesdad

2106 Female 20.976842 1.710730 131.408528 yes yes 3.0 3.0 Sometimes no 1.728139 no
1.676269 0.906247 Sometimes Public_Transportation Obesity_Type_III

2107 Female 21.982942 1.748584 133.742943 yes yes 3.0 3.0 Sometimes no 2.005130 no
1.341390 0.599270 Sometimes Public_Transportation Obesity_Type_III

2108 Female 22.524036 1.752206 133.689352 yes yes 3.0 3.0 Sometimes no 2.054193 no
1.414209 0.646288 Sometimes Public_Transportation Obesity_Type_III

2109 Female 24.361936 1.739450 133.346641 yes yes 3.0 3.0 Sometimes no 2.852339 no
1.139107 0.586035 Sometimes Public_Transportation Obesity_Type_III

2110 Female 23.664709 1.738836 133.472641 yes yes 3.0 3.0 Sometimes no 2.863513 no
1.026452 0.714137 Sometimes Public_Transportation Obesity_Type_III

INPUT: Encode Categorical Variables

```
# Initialize LabelEncoder
```

```
le = LabelEncoder()
```

```
target_names = []
```

```
print("LabelEncoder initialized")
```

```
# Apply encoding to categorical columns
```

```
for col in data.columns:
```

```
if data[col].dtype == 'object':
```

```
data[col] = le.fit_transform(data[col])
```

```
if col == 'NObeyesdad':  
  
target_names = list(le.classes_)  
  
print("Categorical variables encoded")
```

OUTPUT:

LabelEncoder initialized

Categorical variables encoded

INPUT: Split Data

```
# Split data into training and testing sets  
  
X = data.drop('NObeyesdad', axis=1)  
  
y = data['NObeyesdad']  
  
X_train, X_test, y_train, y_test = train_test_split(X, y,  
  
test_size=0.2,  
  
random_state=42)  
  
print("Data split complete")
```

OUTPUT:

Data split complete

GINI

INPUT: Train Decision Tree

```
# Initialize and train Decision Tree with Gini impurity  
  
model = DecisionTreeClassifier(criterion='gini')
```

```
model.fit(X_train, y_train)
```

```
print("Model trained using Gini impurity")
```

OUTPUT:

Model trained using Gini impurity

INPUT: Evaluate Model

```
# Evaluate Gini model
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy (Gini, Full Tree):", accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix (Gini, Full Tree):\n", confusion_matrix(y_test, y_pred))
```

OUTPUT:

Accuracy (Gini, Full Tree): 0.9432624113475178

Confusion Matrix (Gini, Full Tree):

```
[[54 2 0 0 0 0 0]
```

```
[ 5 55 0 0 0 2 0]
```

```
[ 0 1 72 3 0 0 2]
```

```
[ 0 0 3 55 0 0 0]
```

```
[ 0 0 0 0 63 0 0]
```

```
[ 0 4 0 0 0 52 0]
```

```
[ 0 0 0 0 0 2 48]]
```

INPUT: Plot Confusion Matrix

```
# Compute confusion matrix for Gini

cm = confusion_matrix(y_test, y_pred)

# Plot confusion matrix (Gini)

plt.figure(figsize=(8,6))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

xticklabels=target_names, yticklabels=target_names)

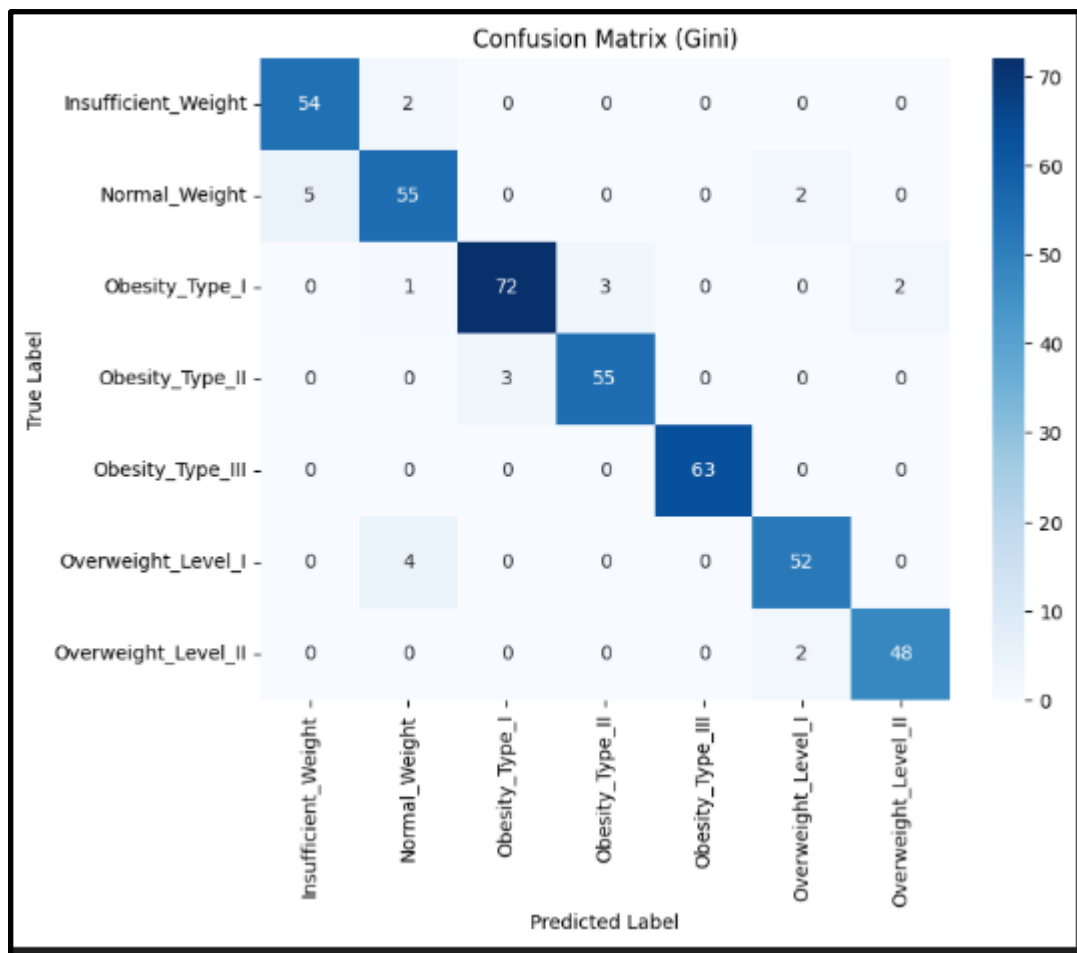
plt.title('Confusion Matrix (Gini)')

plt.ylabel('True Label')

plt.xlabel('Predicted Label')

plt.show()
```

OUTPUT:



ENTROPY

INPUT: Train Decision Tree

Train and evaluate Decision Tree with Entropy

```
model_entropy = DecisionTreeClassifier(criterion='entropy')
```

```
model_entropy.fit(X_train, y_train)
```

```
y_pred_entropy = model_entropy.predict(X_test)
```

```
print("Accuracy (Entropy, Full Tree):", accuracy_score(y_test, y_pred_entropy))
```

```
print("\nConfusion Matrix (Entropy, Full Tree):\n",
```

```
confusion_matrix(y_test, y_pred_entropy))
```

OUTPUT:

Accuracy (Entropy, Full Tree): 0.9598108747044918

Confusion Matrix (Entropy, Full Tree):

```
[[53 3 0 0 0 0]
```

```
[ 5 56 0 0 0 1]
```

```
[ 0 0 76 2 0 0]
```

```
[ 0 0 2 56 0 0]
```

```
[ 0 0 0 0 63 0]
```

```
[ 0 0 0 0 0 56]
```

```
[ 0 0 1 0 0 3 46]]
```

INPUT: Evaluate Model

```
# Compute confusion matrix for Entropy
```

```
print("Calculating confusion matrix (Entropy)...")
```

```
cm_entropy = confusion_matrix(y_test, y_pred_entropy)
```

OUTPUT:

Calculating confusion matrix (Entropy)...

INPUT: Plot Confusion Matrix

```
# Plot confusion matrix (Entropy)
```

```
print("Displaying plot...")
```

```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(cm_entropy, annot=True, fmt='d', cmap='Greens',
```

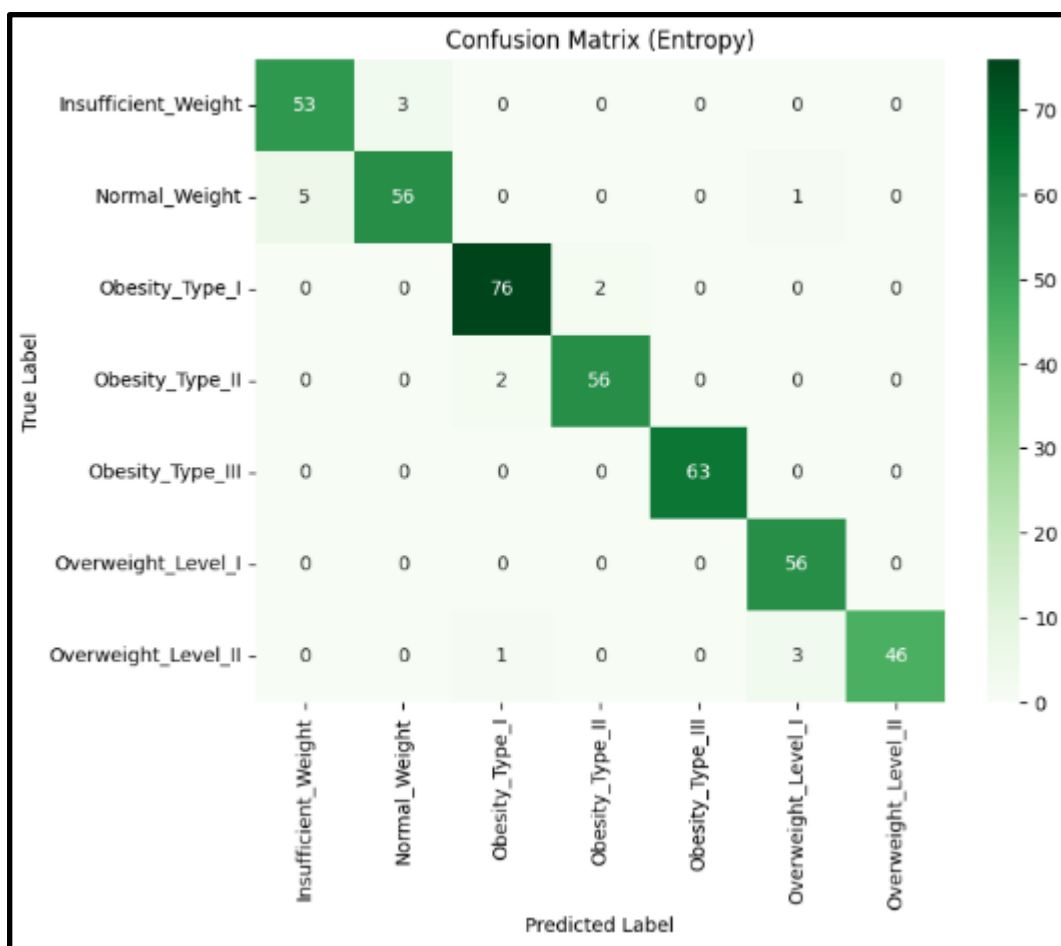
```
xticklabels=target_names, yticklabels=target_names)
```

```
plt.title('Confusion Matrix (Entropy)')
```

```
plt.ylabel('True Label')
```

```
plt.xlabel('Predicted Label')
```

```
plt.show()
```



INPUT: GINI(MAX_DEPTH=4)

```
# Set maximum depth
```

```
d = 4
```

```
# Initialize Decision Tree with max depth = 4

clf = DecisionTreeClassifier(max_depth=d, criterion='gini', random_state=42)

# Train the model

clf.fit(X_train, y_train)

# Evaluate the model

acc = accuracy_score(y_test, clf.predict(X_test))

print(f"Max Depth: {d}, Accuracy: {acc:.4f}")

# Plot the Decision Tree

plt.figure(figsize=(20,10))

plot_tree(

    clf,

    feature_names=X.columns,

    class_names=target_names,

    filled=True,

    rounded=True

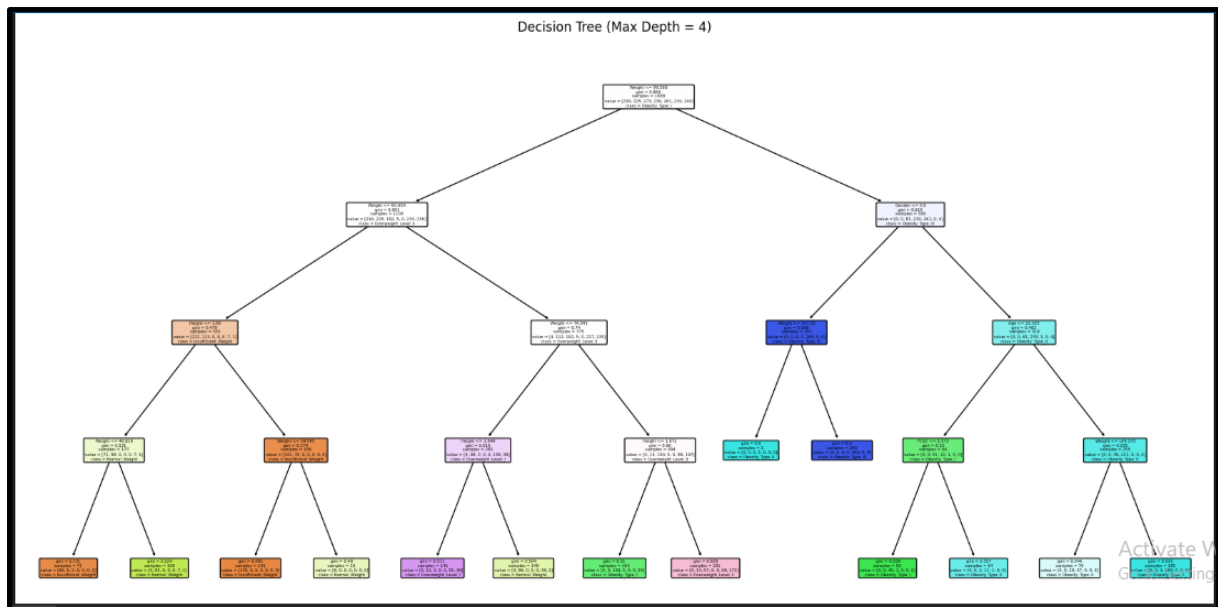
)

plt.title(f"Decision Tree (Max Depth = {d})")

plt.show()
```

OUTPUT

Max Depth: 4, Accuracy: 0.7541



INPUT: ENTROPY(MAX_DEPTH=4)

Analyze Decision Tree with Max Depth = 2 (Entropy)

d = 2

Initialize Decision Tree with entropy criterion

clf = DecisionTreeClassifier(

max_depth=d,

criterion='entropy',

random_state=42

)

Train the model

clf.fit(X_train, y_train)

Evaluate the model

acc = accuracy_score(y_test, clf.predict(X_test))

```
print(f"Max Depth (Entropy): {d}, Accuracy: {acc:.4f}")
```

```
# Plot the Decision Tree
```

```
plt.figure(figsize=(20,10))
```

```
plot_tree(
```

```
clf,
```

```
feature_names=X.columns,
```

```
class_names=target_names,
```

```
filled=True,
```

```
rounded=True
```

```
)
```

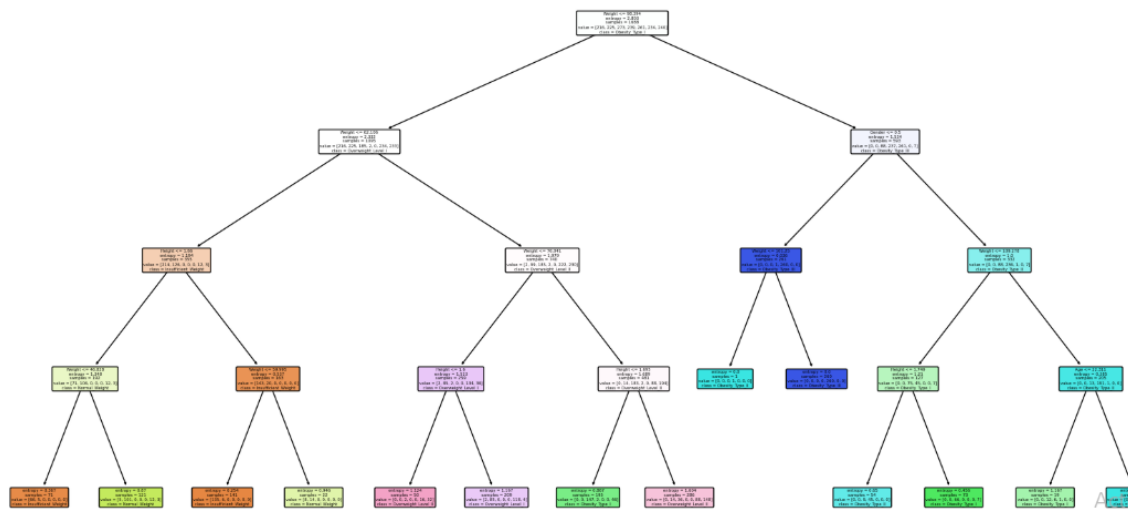
```
plt.title(f"Decision Tree (Max Depth = {d}, Entropy)")
```

```
plt.show()
```

OUTPUT:

Max Depth (Entropy): 4, Accuracy: 0.7400

Decision Tree (Max Depth = 4, Entropy)



Go to Setting